
Paralleling Human Neuroplasticity through Transformers

Shreya Shaji, Anya Vaish, Vrinda Rathi
University of Washington
{sshaji, avaish, vrathi23}@uw.edu

Machine Learning Project Summary

Project Scope

This project aimed to evaluate whether neuroplasticity-inspired parameter management can reduce catastrophic forgetting in transformer-based continual learning. Inspired by the TriRE framework, we tested whether preserving important weights and selectively resetting remaining weights would improve accuracy across sequential tasks compared to standard fine-tuning and replay-based baselines. We investigated three hypotheses: (1) preserving important weights would improve performance on previous tasks, (2) resetting unimportant weights and retraining would allow for new representations while retaining prior knowledge, and (3) checkpoints incorporating active forgetting would outperform those using only weight preservation. Our evaluation used average accuracy and forgetting across five sequential tasks.

Methodology

We implemented our approach in PyTorch, using the BERT-tiny transformer model. The training procedure was inspired by the TriRE continual learning framework and Lottery Ticket Hypothesis literature. After training on each task, we calculated a Continual Weight Importance (CWI) score using the weight magnitude and gradient contributions to current and previous tasks. The top 50% of weights were considered important, while the rest were considered unimportant. These groups were then retrained or reset differently across four checkpoint variants. The experiments were conducted on five sequential text-classification datasets from the Hugging Face Dataset library: SST-2, AG News, Emotion, DBPedia-14, and Yahoo Answers. All code was written from scratch and executed on a MacBook Pro with an Apple M2 Max GPU, with all experiments and comparisons completing in approximately 2 hours.

Results

Our results showed that the neuroplasticity-inspired training approach did not significantly outperform replay-based continual learning baselines. The best checkpoint models achieved approximately 0.77 average accuracy, while the Memory Replay and Equal-Compute Replay baselines achieved roughly 0.78. Forgetting rates were also comparable across approaches, with replay-based methods slightly outperforming the checkpoint models. Checkpoints that incorporated active weight resetting (CP3 and CP4) consistently performed slightly better than those without resetting (CP2), when trained with more epochs and higher importance thresholds. These results suggest that while neuroplasticity-inspired parameter management can produce competitive results, it does not provide significant advantages over simple replay-based approaches when applied to small, pretrained transformers such as BERT-tiny.

What was Easy

The project was simplified by the clear structure provided by the TriRE framework and related continual learning literature. These resources provided guidance for designing the training pipelines and designating important parameters. PyTorch also made it straightforward to implement gradient-based importance scoring, masking, and weight isolation techniques.

What was Difficult

One challenging aspect of this project was building and evaluating a continual learning system across multiple classification tasks with different-sized output spaces. We also went through an iterative process of choosing meaningful metrics and ensuring fair comparisons across training procedures, which required significant experimentation and debugging.

1 Introduction

Large neural networks are often highly overparameterized, and prior work has shown that transformer models can be pruned substantially while maintaining competitive performance [4, 3, 9]. Additionally, lottery-ticket-style results suggest that useful behavior may often be driven by a smaller subnetwork of parameters rather than by all weights equally [10]. These findings motivate a broader question beyond compression: if some parameters matter more than others, can selectively preserving and reusing them help a model retain knowledge over time?

This question is especially relevant in continual learning, where models are trained on tasks sequentially and often suffer from catastrophic forgetting, meaning that performance on earlier tasks degrades as new tasks are learned. Continual learning can also be viewed as a stability-plasticity problem, where a model must remain stable enough to retain prior knowledge while also staying plastic enough to adapt to new tasks. Recent work such as TriRE has explored neuroplasticity-inspired approaches for continual learning by identifying important parameters and reducing interference between old and new knowledge [8]. Related neuroscience-inspired and machine learning work likewise suggests that selective strengthening, forgetting, and relearning may be useful mechanisms for balancing memory retention with adaptation [6, 5].

Motivated by these ideas, our project studies whether a neuroplasticity-inspired parameter-management strategy can improve continual learning in transformers. Rather than treating all model weights equally during sequential learning, we investigate whether identifying and selectively preserving important parameters can improve accuracy retention across tasks compared to standard sequential fine-tuning.

2 Scope of the Project

Studies have shown that large neural networks can be pruned to a smaller network of weights and achieve competitive accuracy with larger models [2]. This suggests that a smaller subnetwork of weights is what is driving the accuracy. Building on this, parameter isolation and weight-management strategies that preserve these important weights have been proven to increase accuracy across several tasks in other architectures such as DNNs [8]. These strategies emulate the brain’s processes for selective synaptic strengthening. This project tests whether applying parameter isolation strategies paralleling neuroplasticity using weight importance scoring and selective retraining to a BERT-tiny transformer improves average accuracy retention across 5 sequential classification tasks compared to standard sequential fine-tuning. We hypothesize that preserving and reinforcing the weights identified as important and resetting unimportant weights to pretrained values will result in higher accuracy on previously seen tasks without decreasing performance on the current task compared to a sequentially trained baseline model. This is because it allows for the model to reinforce important relationships related to accuracy and form new important relationships after weights are reset.

2.1 Addressed Hypotheses

1. Preserving weights identified as important and reinforcing them on task-specific data will result in higher accuracy on previously seen tasks compared to standard sequential fine-tuning.
2. Resetting unimportant weights to pretrained values and retraining on combined task and memory data will allow for the model to form new connections, improving accuracy across tasks.
3. Checkpoints that incorporate active weight resetting (CP3, CP4) will exhibit higher accuracy than the study-only checkpoints (CP2) by allowing unimportant weights to form better connections from a reset starting point in the middle of fine-tuning.

3 Methodology

Code for our project is available at <https://github.com/sshaji03/transformer-neuroplasticity>.

In order to parallel neuroplasticity within transformers, we drew inspiration from the paper “TriRE: A Multi-Mechanism Learning Paradigm for Continual Knowledge Retention and Promotion” by Preetha Vijayan et. al. This paper explores the issue of catastrophic forgetting in DNN continual learning tasks. Continual learning is when a machine learning model is introduced to different tasks sequentially and the goal is for the model to learn how to achieve high accuracy in all of these tasks. A primary roadblock to this is catastrophic forgetting, where the model rapidly loses accuracy for previous tasks when a new task is introduced because it is too focused on the task it’s currently training on. This paper is inspired by the brain’s methods of learning and this motivates their methods of identifying important weights to a task and training important and unimportant weights based on different datasets to prevent catastrophic forgetting by

allowing important weights to study task material, and utilizing unused weights to find new connections and remember old tasks. This paper’s methodology is where we derived our own process from [8].

In our study, we aim to study how applying principles of neuroplasticity to transformer training will impact its accuracy on continual learning tasks. Our method implements identifying certain weights in the transformers as important to the task and previous tasks and the rest of the weights are separated as unimportant. Throughout our training loop, the model at different checkpoints is saved so we can evaluate the effectiveness of different neuroplasticity principles on model accuracy. To model studying, we allow for certain weights we identify to be important to study the task-specific data again, to fine-tune. To model active forgetting, we allow for unimportant weights to reset to the model’s default value and train our current and past task’s data. This is the method for forgetting unimportant ideas and learning new ideas. The model is also able to train its unimportant weights only on items that the model must retain memory of (previously learned tasks). This separation of weight training allows us to emulate the principles of neuroplasticity constructively.

In order to calculate which weights we designated as being important and which ones we designated as being unimportant we used a scoring based on the TriRE paper. This paper uses a Continual Weight Importance metric that takes into account the weight’s current value and importance in previous and current tasks. We implemented our version of this by taking an even percentage of a weight’s magnitude, their impact on the current task’s loss, and their averaged impact on previous tasks loss to compute a score. The weights within the top 50 percent of continual weight importance score are designated important [8]. In order to train the weights separately we use the method described in the paper “Deconstructing Lottery Tickets: Zeros, Signs, and the Supermask” by Hattie Zhou et. al. This paper describes how to prune weights from a network that was originally larger by setting them equal to 0 and using a gradient hook [11]. We utilized this masking and gradient hook method to separate our different weight sets.

To conduct our experiment we used the BERT-tiny model class. This model class utilizes 4.4 million parameters, 2 layers, and a hidden dimension size of 128 [1, 7]. This lightweight model is meant to be a base model for classification tasks. This pretrained model’s efficiency makes it a good base for our model to conduct several fine-tuning tasks in a reasonable time. For every training task, the model trains over 3 epochs, with a learning rate of 0.001 and a seed of 42 for any random tasks.

We defined 5 learning tasks for our continual learning. We used Hugging Face’s Dataset library to define 5 datasets. The SST-2 dataset contains sentences with negative and positive labels. The AG-News dataset has news story titles and a label of whether the article falls under “world”, “sports”, “business”, and “science.” The Dair-Ai Emotion dataset has tweets and a label of the underlying emotion out of 6 choices. The DBpedia-14 dataset is made up of wikipedia articles and 14 content labels. Finally, the Yahoo Answers dataset contains questions asked on Yahoo and 10 different labels for topics. For each dataset, 5000 of the datapoints from the train set make up the task-specific dataset. 500 different datapoints from the train set are added to the memory dataset that is used during training. 500 datapoints from the test dataset of each task make up the test data. Since each of these tasks have a different number of classes, we have different last layer heads for each task that we will switch when the datapoint is from a certain task just for the last layer of classification. The underlying weight used for all the models stays the same.

The training loop loops over each task one at a time in the above order. There are 4 phases of training the model, each time the data is trained over 3 epochs, which is sufficient for fine-tuning, using an AdamW optimizer. It first trains the entire model on the training data for the specific task. Then it computes the CWI score. Then, using the CWI score, we designate the top 50 percent of those weights to be important weights and the other 50 percent are unimportant weights. This designation is only in the scope of this current task. This is used to create gradient hooks and masks on the important versus unimportant weights so that in the future the updating can be done separately and merged together at a later checkpoint. The model with only the important weights is saved for checkpoint 1 (CP1).

For checkpoint 2 (CP2), the important weights are trained on the task training data and the unimportant weights are trained on the memory data. This memory data is an equal representation of previous tasks. This is to represent the model being able to study without dealing with catastrophic forgetting.

For checkpoint 3 (CP3), the same model of important weights from CP1 are used. Then the unimportant weights are reset to their initial value and the entire model is trained on the task’s data and the memory. This represents the model discarding task-specific fine-tuning in the unimportant weights to allow new connections to form, testing whether this improves overall accuracy. The values are reset to the BERT model’s initial values so it starts from the same starting point that BERT had.

The final checkpoint (CP4) uses the important weights from checkpoint 2 and does the same unimportant weight resetting and then trains on the entire task data and the memory. This is to see if the model is able to increase its overall performance when it was given the opportunity to study or revise the task data in CP2.

In order to assess the impact of our experimental training loop we focused on the accuracy and loss of the model as it learned over more tasks and also evaluated the model’s forgetting. Forgetting was defined as how accuracy on a certain task dropped after a new task was learned. We implemented different baseline models to compare our checkpoints. The normal is for when the model trains over the number of epochs once only using training data at each task. The Memory Replay model follows the same protocol for epochs but introduces the memory data into each task’s training. The Equal-Compute Replay model trains for the same number of epochs that checkpoint 4 has trained in the neuroplasticity model while introducing memory data into training. The Post-training (neuro) checkpoint is the model after checkpoint 1 if the unimportant weights were not zeroed out. It’s identical to the normal baseline model, just produced by the neuroplasticity training loop.

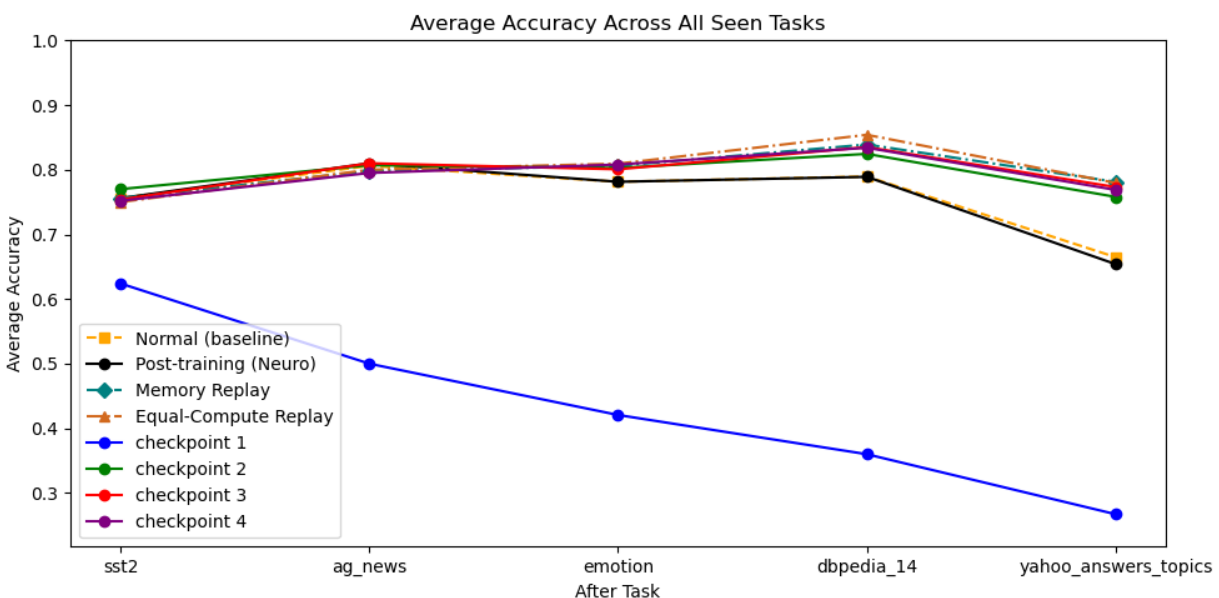
The model’s accuracy and loss at each checkpoint across the tasks is evaluated against the test data of tasks it has seen before to see where accuracy and loss are affected compared to a training loop where the model trains each task once sequentially.

All code was written from scratch using the TriRE and Lottery Ticket papers as methodological references. The experiments were run on a MacBook Pro with an Apple M2 Max Chip and 32 GB of memory. The code utilizes Apple’s Metal Performance Shaders for GPU acceleration. The experiment with neuroplasticity modifications and all baseline training runs took 2 hours to complete.

4 Results/Summary

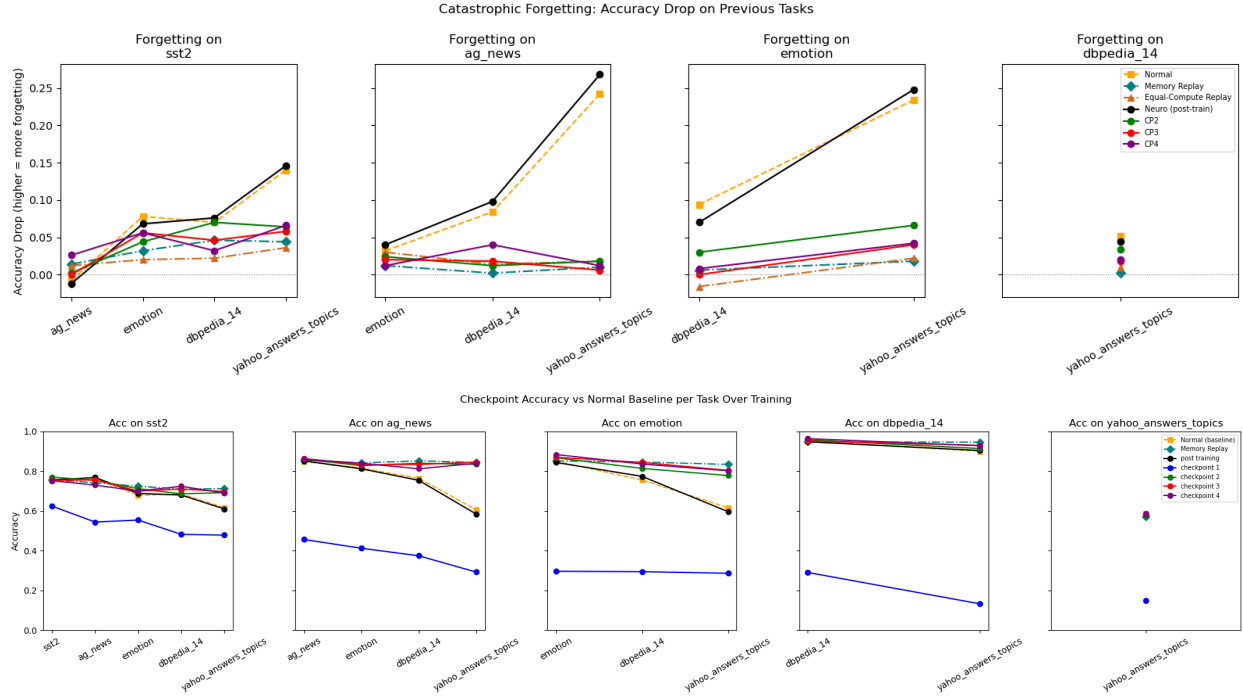
We evaluated the neuroplasticity-inspired training model approach across 5 sequential text classification tasks, measuring average accuracy retention and catastrophic forgetting across four checkpoints (CP1-CP4) and compared them to 3 baselines: Normal, Memory Replay, and Equal-Compute Replay models. We conducted a hyperparameter sweep across the number of epochs (3, 5, 10) and the weight importance threshold k (0.1, 0.2, 0.3, 0.5). Each configuration was evaluated on average accuracy on seen tasks and per-task accuracy drop after learning new tasks. The checkpoint models did not achieve a higher average accuracy than the Memory Replay and Equal-Compute Replay models, with the checkpoints reaching a maximum accuracy of 0.77 compared to 0.78 accuracy for the other two baselines. The checkpoint models also did not display a lower level of forgetting because the lowest amount of forgetting the checkpoint models achieved was 0.03 and the two baselines achieved a minimum of 0.02 forgetting. The checkpoint models within values of k showed an increase of average accuracy and decrease of forgetting as the number of epochs increased. Checkpoints with active weight resetting like CP3 and CP4 showed marginally higher accuracy than CP2 (0.77 vs. 0.76) at higher epoch counts and k values.

4.1 Result 1: Average accuracy and forgetting does not improve in checkpoint models compared to baseline with memory data



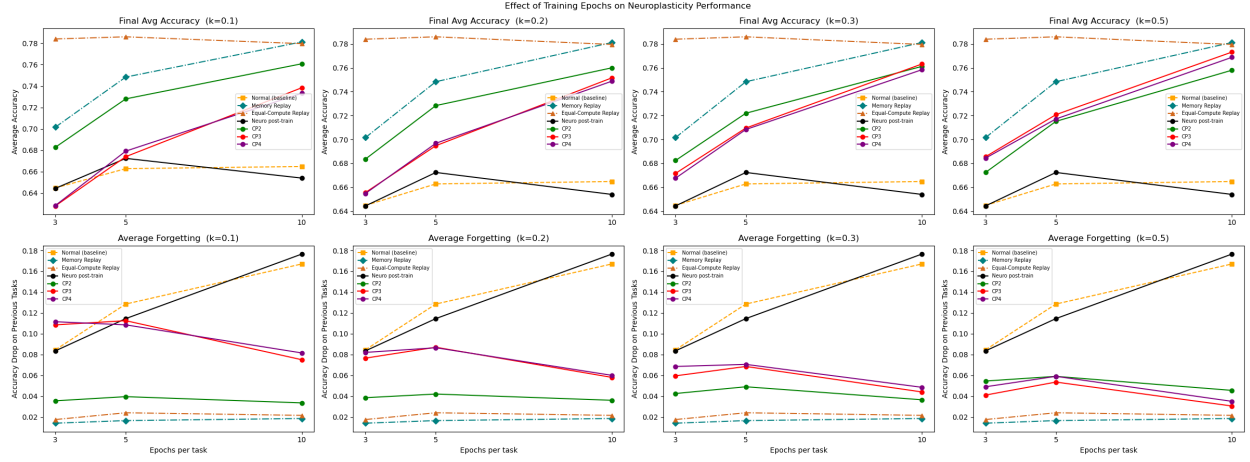
The first graph compares average accuracy across all seen tasks for each model over the course of continual learning for

the 10 epochs $k = 0.5$ configuration. This model was chosen because in the hyperparameter sweep the accuracy of the checkpoints was the most competitive to the Memory Replay and Equal-Compute Replay models. The first checkpoint shows a trend in declining accuracy, dropping from 0.62 after the first task to 0.28 by the final task. The Normal and the post-training model (the model right after checkpoint 1 with none of the weights zeroed out) exhibited lower accuracy, 0.66 and 0.65 respectively, than the checkpoint models. These exhibited 0.76 for CP2, 0.77 for CP3, and 0.77 for CP4. The memory replay and equal compute replay models exhibit very similar accuracy to the checkpoints, and they achieve a final average of 0.78. They have a minutely higher accuracy compared to the checkpoint models at the last two tasks. The gap is approximately 0.01-0.02. This graph does not support hypothesis one because none of the checkpoint models achieve a higher accuracy compared to the Memory Replay and Equal Compute models, which are slightly more accurate.



This graph displays the forgetting and accuracy as new tasks are introduced. The general trend for non-checkpoint models is that accuracy decreases and forgetting increases on each task as new tasks are introduced. For the equal-compute replay model, the forgetting on sst2 increases from 0.01 to 0.03 by the end of training on all 5 tasks. For CP2, CP3 and CP4 there is no clear trend for accuracy or forgetting as they fluctuate across continual learning tasks. There is no clear trend in which model has a higher accuracy or lower forgetting because at different points in time different models achieve higher accuracy or lower forgetting. CP1 consistently has the lowest accuracy over different tasks, an example being declining from 0.45 to 0.30 on AG News by the final task. The Equal Compute Replay and Memory Replay models are competitively accurate compared to CP2, CP3, and CP4. It disproves hypothesis 1 since the CP2, CP3, and CP4 were not significantly larger in accuracy than the Equal Compute Replay and Memory Replay model even though they are close in value.

4.2 Result 2: Checkpoints with weight resetting exhibit higher accuracy than checkpoints without weight resetting at higher epochs



The Equal Compute Replay model remained stable in average accuracy and forgetting across different values of epochs. The Memory Replay models both showed that as the number of epochs increased, the accuracy increased but forgetting remained stable. CP2, CP3, and CP4 increased in accuracy as the number of epochs increased but forgetting was stable for every possible value of k . The forgetting for every value of k increased from 3 to 5 epochs and decreased from 5 to 10 epochs for CP2, CP3, and CP4. At 10 epochs and $k=0.5$, the Equal-Compute Replay model achieves only marginally higher accuracy, around 0.78, than CP3/CP4, around 0.77, at the final task, whereas at 3 epochs the gap is larger: 0.78 vs 0.69. This relates to hypothesis 3 as the higher number of epochs resulted in an overall higher accuracy for CP3 and CP4 but this did not hold for CP2.

4.3 Result 3: Checkpoints with weight resetting improve relative to CP2 at higher importance thresholds

CP3 and CP4 show lower accuracy for k values of 0.1, 0.2 and 0.3 compared to CP2, but have a higher accuracy than CP2 for $k = 0.5$. The overall accuracy of CP2 stayed stable at $k = 0.1$ and 0.2 but drops as k increases after this. The overall accuracy for CP3 and CP4 increases as k increases. We see a similar pattern in the forgetting where the forgetting rate increases for CP2 at $k = 0.3$ and 0.5 and the forgetting rate steadily decreases as k increases for CP3 and CP4. This relates to hypothesis 2 as this evidence indicates under a higher number of epochs CP3 and CP4 did display a higher accuracy and lower forgetting rate. The task specific accuracy and forgetting graphs do not show concrete evidence for hypothesis two since the difference between CP2 and CP3/CP4 since they fluctuate between performing better on specific tasks.

5 Discussion

The results indicate that there is not a significant difference in accuracy and forgetting between CP2, CP3, and CP4 and the models that were trained with memory data for fewer epochs and with memory data for an equal number of epochs in the 10 epochs $k = 0.5$ setting. This indicates that the structure of this transformer does not lend itself to better accuracy or lower forgetting rates with this extra training that incorporates neuroplasticity principles. The model exhibited the same if not higher levels of accuracy across different configurations of epochs for a model that incorporates the memory data into basic training and/or trains across an equivalent number of epochs for the full training loop.

Regarding CP2 vs CP3/CP4, the results showed that when allowed to train for a longer time, the weight resetting in CP3 and CP4 did result in a slightly higher accuracy (CP3/CP4 at 0.77 vs CP2 at 0.76) and lower forgetting compared to CP2. This means that the CP3 and CP4 weight resetting did increase accuracy more at higher levels of epochs, which indicates that the weight resetting could create an advantage.

We are led to believe from the high accuracy numbers that this could be caused by the fine tuning that we are doing. The BERT-tiny model we have is already specialized to this type of training so this specialization in fine-tuning may not create much of an impact. Another possibility is that the model is too small to create a subnetwork of important weights that will allow the model to accurately predict. This can be seen by CP1 having an extremely low accuracy because the small networks were not big enough to predict on the data. Although the results did not support our hypotheses there is still potential for the weight resetting and additional training of unimportant weights to create a meaningful impact on

accuracy for tasks that have more specialized learning requirements. We saw CP3 and CP4 accuracy increase as the value of k went up compared to CP2 accuracy. This means that the weight resetting allowed CP3 and CP4 to increase their accuracy in a way that's not correlated to purely more epochs since at lower values of k they were not able to achieve this. However, this could potentially be driving it to be more similar to the model that was trained with memory data by allowing more of the neurons to be trained on the task specific data. As k increases, more of the neurons are exposed to the brunt of the data used to train for a specific task, so it is likely that this increase in accuracy can be attributed to this, and the benefits to this can be seen in the Memory Replay and Equal Compute Replay models.

5.1 What was Easy

The TriRE framework gave our experiment a good basis for evaluating the importance of different weights and gradients within the training. This made our initial loops to test the different checkpoints easy to set up. The TriRE paper provided an algorithm that showed how the data looped through different training architectures and separated data. This made it easy to conceptualize what the training process would look like [8]. In order to actually implement the process, the PyTorch library made it simple to evaluate the gradient and weight values at different points in the training. Previous knowledge of PyTorch greatly contributed to this as well, as our team had experience with isolating gradients. PyTorch allowed our model to access values to calculate the CWI and also use gradient hooks to train our weights. The documentation on how to train subnetworks by setting all other weights to 0 helped with implementing isolation of training different networks.

5.2 What was Difficult

Since the training data is simulating a continual learning task, simulating this with a transformer with different tasks sequentially was a bit difficult. One challenge we faced was how to use the same underlying model weights for the different classification tasks with a different number of outputs. We solved this by using the same underlying weights but changing the final layer's head to correspond with how many outputs the class had. This way the weights would be applied to all tasks but the output would still be valid. This was difficult because working with BERT-tiny for the first time, understanding how to manipulate the layers required previous knowledge of how the BERT-tiny outputted its final classification. Another challenge was defining the attributes that we could measure to assess the different models. For example, the accuracy was what we first measured but in order to assess catastrophic forgetting we must look at the accuracy drop specifically. Assessing averages versus over training certain tasks was also important. Implementing these assessments required additional architectural planning because we lacked experience assessing a model on anything other than accuracy or loss. Figuring out how to account for more epochs resulting in a higher accuracy and lower forgetting was also a learning curve. We implemented several baseline models for this reason, and this required evaluating all factors that could lead to improved accuracy and isolating different improvements in these baseline models to be able to properly evaluate.

5.3 Recommendations for Future Work

In the future, we would recommend that this idea should be explored with an original model. Our results indicate that the specific fine-tuning for this BERT-tiny model, the specialized training that emulated neuroplasticity did not create a significant difference compared to just introducing the memory to the training data. However, there is still a potential that this difference in training does create a difference when the model is being trained from scratch on continual learning tasks. This would require a higher compute power, as working to train a model from scratch requires more epochs. This paper can act as a reference for which hyperparameters to focus on and a training architecture, but the model could be built from the ground up. Additionally, using data classes that require specialization could also allow this architecture to have an interesting effect on the accuracy as it allows for more room for specialization. We also recommend exploring new forms of weight importance scoring as this impacts how the important subnetworks are created. If the important subnetworks are better identified this could impact overall accuracy.

References

- [1] Prajjwal Bhargava, Aleksandr Drozd, and Anna Rogers. Generalization in nli: Ways (not) to go beyond simple heuristics, 2021.
- [2] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Training pruned neural networks. *CoRR*, abs/1803.03635, 2018.
- [3] Woosuk Kwon, Sukmin Kim, Michael W. Mahoney, Jihun Hassoun, Kurt Keutzer, and Amir Gholami. A fast post-training pruning framework for transformers. In *Advances in Neural Information Processing Systems*, 2022.
- [4] Paul Michel, Omer Levy, and Graham Neubig. Are sixteen heads really better than one? In *Proceedings of the 33rd Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- [5] Tom J. Ryan and Paul W. Frankland. Forgetting as a form of adaptive engram cell plasticity. *Nature Reviews Neuroscience*, 23:173–186, 2022.
- [6] Anni Sha, Bruno P. Nunes, and Armin Haller. “Forgetting” in Machine Learning and Beyond: A Survey. *arXiv preprint arXiv:2405.20620*, 2024.
- [7] Iulia Turc, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Well-read students learn better: The impact of student initialization on knowledge distillation. *CoRR*, abs/1908.08962, 2019.
- [8] Preetha Vijayan, Prashant Bhat, Elahe Arani, and Bahram Zonooz. Trire: A multi-mechanism learning paradigm for continual knowledge retention and promotion, 2023.
- [9] Mengzhou Xia, Zexuan Zhong, and Danqi Chen. Structured pruning learns compact and accurate models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, 2022.
- [10] Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. In *Advances in Neural Information Processing Systems*, 2019.
- [11] Hattie Zhou, Janice Lan, Rosanne Liu, and Jason Yosinski. Deconstructing lottery tickets: Zeros, signs, and the supermask. *CoRR*, abs/1905.01067, 2019.